

1. Error Handling in C#

Error handling in C# is primarily done through **exceptions**. Exceptions are unexpected events that occur during the execution of a program, and C# provides a structured way to handle these exceptions using `try`, `catch`, and `finally` blocks.

1.1 Checked and Unchecked Statements

- **Checked Statements:** These are used to explicitly check for overflow in arithmetic operations. If an overflow occurs, a `System.OverflowException` is thrown.

```
csharp
VerifyOpen In EditorRunCopy code
1checked
2{
3    int maxValue = int.MaxValue;
4    // This will throw an OverflowException
5    int result = maxValue + 1;
6}
```

- **Unchecked Statements:** These allow arithmetic operations to overflow without throwing an exception. The overflow is ignored, and the result wraps around.

```
csharp
VerifyOpen In EditorRunCopy code
1unchecked
2{
3    int maxValue = int.MaxValue;
4    // This will not throw an exception; result will wrap around
5    int result = maxValue + 1;
6}
```

2. Exception Handling Structure

2.1 try, catch, finally

- **try:** Contains the code that may throw an exception.
- **catch:** Catches and handles the exception.
- **finally:** Executes code after the try/catch blocks, regardless of whether an exception was thrown.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2
3public class ExceptionHandlingExample
4{
5    public static void Main()
6    {
7        try
8        {
9            Console.WriteLine("Enter a number: ");
10           int number = Convert.ToInt32(Console.ReadLine());
11           Console.WriteLine($"You entered: {number}");
12        }
13        catch (FormatException e)
```

```

14      {
15          Console.WriteLine($"Invalid input: {e.Message}");
16      }
17      catch (Exception e)
18      {
19          Console.WriteLine($"An error occurred: {e.Message}");
20      }
21      finally
22      {
23          Console.WriteLine("Execution completed.");
24      }
25  }
26}

```

3. Best Practices for Exception Handling

3.1 Dos

- **Use Specific Exceptions:** Catch specific exceptions to handle errors more effectively.
- **Log Exceptions:** Always log exception details for debugging.
- **Use `finally` for Cleanup:** Ensure resources are released in the `finally` block.
- **Throw Exceptions Appropriately:** Use `throw;` to preserve the stack trace.
- **Create Custom Exceptions:** Define custom exceptions for specific error scenarios.
- **Use Exception Filters:** Catch exceptions based on conditions.
- **Centralized Exception Handling:** Implement a consistent error handling mechanism.

3.2 Don'ts

- **Don't Catch General Exceptions:** Avoid catching `System.Exception` unless necessary.
- **Don't Swallow Exceptions:** Always handle exceptions appropriately.
- **Don't Use `throw ex;`** This resets the stack trace; use `throw;` instead.
- **Don't Return Error Codes:** Use exceptions instead of error codes.
- **Don't Use Empty Catch Blocks:** Always provide handling logic.
- **Don't Throw Exceptions in Finally Blocks:** This can obscure other exceptions.
- **Don't Use If Conditions for Exception Handling:** Use `try-catch` instead.

4. User-Defined Exception Classes

Creating custom exceptions allows for more specific error handling.

Example:

```

csharp
VerifyOpen In EditorRunCopy code
1using System;
2
3[Serializable]
4public class NotPrimeException : Exception
5{
6    public int NotAPrime { get; }
7
8    public NotPrimeException(int value)
9        : base($"{value} is not a prime number.")
10    {
11        NotAPrime = value;

```

```

12     }
13}
14
15// Usage
16try
17{
18     int number = 4; // Example number
19     if (number < 2 || (number > 2 && number % 2 == 0))
20     {
21         throw new NotPrimeException(number);
22     }
23}
24catch (NotPrimeException e)
25{
26     Console.WriteLine($"{e.NotAPrime} is not prime.");
27}

```

5. Declaring and Raising Events

Events in C# are based on delegates and allow classes to notify other classes when something of interest occurs.

5.1 Declaring Events

- Use the `event` keyword followed by the delegate type.

Example:

```

csharp
VerifyOpen In EditorRunCopy code
1public class Counter
2{
3    public event EventHandler ThresholdReached; // Declare the event
4
5    protected virtual void OnThresholdReached(EventArgs e)
6    {
7        ThresholdReached?.Invoke(this, e); // Raise the event
8    }
9}

```

5.2 Raising Events

- Use a method to raise the event, typically named `On<EventName>`.

Example:

```

csharp
VerifyOpen In EditorRunCopy code
1public void Add(int x)
2{
3    total += x;
4    if (total >= threshold)
5    {
6        OnThresholdReached(EventArgs.Empty); // Raise the event
7    }
8}

```

Conclusion

- Exception handling is vital for building robust applications in C#.

- Use `try`, `catch`, and `finally` to manage exceptions effectively.
- Follow best practices to ensure proper error handling.
- Create user-defined exceptions for specific error scenarios.
- Understand how to declare and raise events for effective communication between classes.

Feel free to ask if you have any specific scenarios or further questions about these topics!